

Legacy Code Analyser

AI-Powered Legacy Java Code Analysis Tool — Setup & Usage Guide

Team: diaX

Team Members:

Arya Aryan (MS/ETM2-MA)

Dasharath Pooja Lakshmi A (MS/EAB1-MA)

Sivakumar Gowtham (MS/ETM1-MA)

1. Overview

Legacy Code Analyser is a web application that uses AI to help developers understand, modernise, and remediate legacy Java codebases. A user pastes code, uploads a single .java file, or uploads a .zip of an entire Java project, and the tool runs it through five analysis modes in one click, returning plain-English explanations, modernised code, identified bugs and security risks, generated documentation, and a prioritised remediation plan.

The tool was built for the FoA9 AI-First Hackathon under the “Understand Legacy Code” track, with a focus on giving engineering teams a fast, structured first pass at unfamiliar legacy systems before manual review.

2. What It Does

Clicking Analyse runs all five modes below sequentially against the same input and merges the results into one view:

Mode	Description
Explain	Plain-English summary, business logic, step-by-step execution trace, and a Health Score (complexity, documentation, modernisation, risk).
Convert	Converts legacy Java to modern Java 17 (records, var, modern Collections, try-with-resources). For multi-file zip uploads, each file is converted independently so output stays correctly split by file.
Find Bugs	Identifies bugs, security vulnerabilities, and code smells with severity ratings, line/method references, and concrete fixes.
Generate Docs	Produces structured Markdown documentation (Overview, Architecture, Method Reference, Data Flow, Known Limitations) plus a function-by-function reference.
Remediation Plan	A 3-sprint prioritised modernisation plan with effort estimates, grouped by priority (critical / high / medium).

For large inputs, code is automatically split into chunks (8,000 tokens each) and the chunks are processed concurrently, then merged back into one coherent result.

3. Tech Stack

Frontend

- React 19 + Vite 8 — UI framework and build tool
- Tailwind CSS 4 — styling
- Framer Motion — animations and transitions
- Monaco Editor (@monaco-editor/react) — in-browser code editor with Java syntax highlighting
- Axios — HTTP client for calling the backend API

Backend

- Python 3 + FastAPI — REST API server
- Uvicorn — ASGI server
- Google Gemini API (gemini-2.5-flash, with automatic fallback to gemini-2.5-flash-lite and gemini-3.1-flash-lite) — the AI model powering all analysis
- tiktoken — token counting for chunking decisions
- python-multipart — handles .zip file uploads
- python-dotenv — loads the Gemini API key from a local .env file

Architecture Pattern

A single-page React frontend communicates with a FastAPI backend over two REST endpoints (/analyse for pasted code or a single file, /analyse-zip for a project archive). The backend extracts, chunks, and sends code to the Gemini API once per mode, parses and repairs the JSON response, merges multi-chunk results, and returns a unified payload to the frontend.

4. Prerequisites

- Python 3.9 or later
- Node.js 18 or later (includes npm)
- A Google Gemini API key, generated at aistudio.google.com/apikey (NOT the Google Cloud Console — that produces a key in the wrong format and will fail with a 401 error)

5. Installation on Windows

Step 1 — Install Python and Node.js (skip if already installed)

Open PowerShell and run:

```
winget install -e --id Python.Python.3.12
winget install -e --id OpenJS.NodeJS.LTS
```

Close and reopen PowerShell after installing, then verify:

```
python --version
node --version
```

Step 2 — Get the project files

Unzip the project archive to a folder of your choice, for example C:\Projects\legacy-analyser. Open PowerShell and navigate into it:

```
cd C:\Projects\legacy-analyser
```

Step 3 — Set up the backend

```
cd backend
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

Open the .env file in the backend folder and set your Gemini API key:

```
GEMINI_API_KEY=AIzaSy...your-key-here
USE_LOCAL_MODEL=false
```

Important: the key must come from aistudio.google.com/apikey and start with “AlzaSy”. A key copied from the Google Cloud Console will not work.

Start the backend (keep this terminal open):

```
python -m uvicorn main:app --reload --port 8000
```

You should see “Uvicorn running on http://127.0.0.1:8000”. If you change the .env file later, stop this process with Ctrl+C and restart it — environment variables are only read on startup.

Step 4 — Set up the frontend

Open a new PowerShell window (leave the backend running) and navigate to the frontend folder:

```
cd C:\Projects\legacy-analyser\frontend
npm install
npm run dev
```

Vite will print a local URL, typically <http://localhost:5173>.

Step 5 — Open the app

Open <http://localhost:5173> in your browser. Paste a Java file or drag in a .java file or a project .zip, then click Analyse.

6. Troubleshooting

Symptom	Likely Cause / Fix
Gemini API error (401)	Wrong key format. Generate a new key at aistudio.google.com/apikey , not the Cloud Console.
No module named uvicorn	The virtual environment isn't activated. Run <code>venv\Scripts\activate</code> first, or recreate it if activation doesn't fix it.
Network Error in the browser	The backend isn't running, or <code>main.py</code> is on a different port than the frontend expects (8000).
npm run dev fails with a permission or missing-binary error	Delete <code>node_modules</code> and <code>package-lock.json</code> , then run <code>npm install</code> again.
Gemini API error (503)	Google's servers are temporarily overloaded. The app automatically retries and falls back to alternate models; if it persists for hours, try again later or check aistudio.google.com for outage notices.
Some analysis tabs are empty after Analyse completes	An individual mode failed for that run (often a transient model issue). The app shows a warning banner naming which mode failed and why — re-running Analyse usually resolves it.

7. Project Structure

```

legacy-analyser/
├── backend/
│   ├── main.py           FastAPI app – /analyse and /analyse-zip endpoints
│   ├── prompts.py       System prompts for all 5 analysis modes
│   ├── chunker.py       Token-aware code splitter
│   ├── llm.py           Gemini API caller, retry/fallback, JSON repair
│   ├── zip_handler.py   Recursive .java extraction from uploaded zips
│   ├── health_score.py  Health score validation and labelling
│   ├── requirements.txt
│   └── .env             Gemini API key goes here
├── frontend/
│   └── src/
│       ├── App.jsx
│       ├── hooks/useAnalyse.js
│       └── components/
│           ├── CodeEditor.jsx
│           ├── OutputPanel.jsx
│           ├── HealthScore.jsx
│           └── Header.jsx

```